



UNIVERSIDAD AUTÓNOMA DE ZACATECAS
“FRANCISCO GARCÍA SALINAS”

UNIDAD ACADÉMICA DE MATEMÁTICAS

Licenciatura en Actuaría

CLUSTER JERÁRQUICO DE LA BASE DE DATOS AMAZON.

Actividad del Módulo 2.

Materia:

Grandes Bases de Datos.

Alumna:

Ana Camila López Ceballos.

Profesor:

Dr. José Raúl Castro Esparza.

Zacatecas, Zac.

2026

1 Introducción

En este trabajo se realizó un análisis de clustering jerárquico utilizando la base de datos proporcionada por el profesor, la cual contiene información de clientes de Amazon. El objetivo del trabajo es agrupar a las personas que tienen comportamientos similares, con la finalidad de identificar qué clientes se parecen más entre sí.

2 Desarrollo del análisis

2.1 Carga de librerías

```
[6]: #CLUSTERING JERÁRQUICO.
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os
```

2.2 Carga de la base de datos

```
[7]: #Carga de la base de datos.
data = pd.read_excel("Amazon.xlsx")
data
```

```
[7]: Unnamed: 0  Velocidad Entrega  Precio  Durabilidad  Imagen Producto  \
0      Adam      205      3      345      235
1      Anna      9      15      315      33
2      Bernard      17      26      285      3
3      Edward      135      5      355      295
4      Emilia      3      45      48      39
..      ...      ...      ...      ...      ...
95     Teofan      3      8      32      25
96     Teofil      305      25      46      24
97     Teofila      1      14      26      25
98      Teon      155      11      335      34
99     Teresa      125      9      45      25

      Valor Educativo  Servicio Retorno  Tamano Paquete  Calidad Producto  \
0      24      23      26      21
1      25      4      42      215
2      43      27      41      26
3      18      23      39      195
4      34      46      225      34
..      ...      ...      ...      ...
95      7      21      42      17
96      33      28      355      26
97      24      27      42      185
98      26      29      42      215
```

99	22	3	3	22
----	----	---	---	----

	Numero Estrellas
0	17
1	28
2	33
3	17
4	43
..	...
95	1
96	45
97	23
98	27
99	18

[100 rows x 10 columns]

En esta parte del código cargué el archivo con el que trabajaré en el análisis. Guardé esta base de datos en una variable a la que llamé “data”.

2.3 Selección de variables numéricas

```
[8]: #Selección de variables numéricas
data_sin_nombres = data.iloc[:,1:10]
data_sin_nombres
```

```
[8]:
```

	Velocidad Entrega	Precio	Durabilidad	Imagen Producto	Valor Educativo	\
0	205	3	345	235		24
1	9	15	315	33		25
2	17	26	285	3		43
3	135	5	355	295		18
4	3	45	48	39		34
..	...	...	...	...		...
95	3	8	32	25		7
96	305	25	46	24		33
97	1	14	26	25		24
98	155	11	335	34		26
99	125	9	45	25		22

	Servicio Retorno	Tamano Paquete	Calidad Producto	Numero Estrellas
0	23	26	21	17
1	4	42	215	28
2	27	41	26	33
3	23	39	195	17
4	46	225	34	43
..	...	...	...	...
95	21	42	17	1
96	28	355	26	45

97	27	42	185	23
98	29	42	215	27
99	3	3	22	18

[100 rows x 9 columns]

Una vez cargada la base de datos, separé la columna donde aparecen los nombres de los clientes y conservé solo las variables numéricas. Esto fue necesario, ya que para formar los clústeres solo necesitamos trabajar con los valores de sus características y evaluaciones; después haremos la respectiva identificación con los nombres correspondientes.

2.4 Normalización de los datos

```
[9]: #Normalización de los datos
from sklearn.preprocessing import normalize
data_scaled = normalize(data_sin_nombres)
data_scaled[:5]
```

```
[9]: array([[0.43826336, 0.00641361, 0.73756517, 0.50239946, 0.05130888,
            0.04917101, 0.05558462, 0.04489527, 0.03634379],
            [0.02323527, 0.03872544, 0.81323434, 0.08519598, 0.06454241,
            0.01032679, 0.10843125, 0.55506471, 0.0722875 ],
            [0.05723452, 0.08753514, 0.95951982, 0.01010021, 0.14476966,
            0.09090188, 0.13803618, 0.08753514, 0.11110229],
            [0.25885639, 0.00958727, 0.68069644, 0.56564916, 0.03451419,
            0.04410146, 0.07478074, 0.37390368, 0.03259673],
            [0.01197502, 0.17962533, 0.19160036, 0.15567529, 0.13571692,
            0.18361701, 0.89812667, 0.13571692, 0.17164198]])
```

En esta parte del código normalicé las observaciones para que pudieran compararse de una manera más adecuada.

Como la base de datos tiene muchos clientes, en lugar de mostrar todos los resultados, decidí visualizar solo los primeros para comprobar que la normalización se había realizado correctamente.

2.5 Conversión de los datos normalizados a DataFrame

```
[10]: #Datos normalizados en DF
data_scaled = pd.DataFrame(data_scaled, columns= data_sin_nombres.columns)
data_scaled.head()
```

```
[10]:  Velocidad Entrega    Precio  Durabilidad  Imagen Producto  Valor Educativo \
0          0.438263  0.006414    0.737565          0.502399    0.051309
1          0.023235  0.038725    0.813234          0.085196    0.064542
2          0.057235  0.087535    0.959520          0.010100    0.144770
3          0.258856  0.009587    0.680696          0.565649    0.034514
4          0.011975  0.179625    0.191600          0.155675    0.135717
```

Servicio Retorno Tamano Paquete Calidad Producto Numero Estrellas

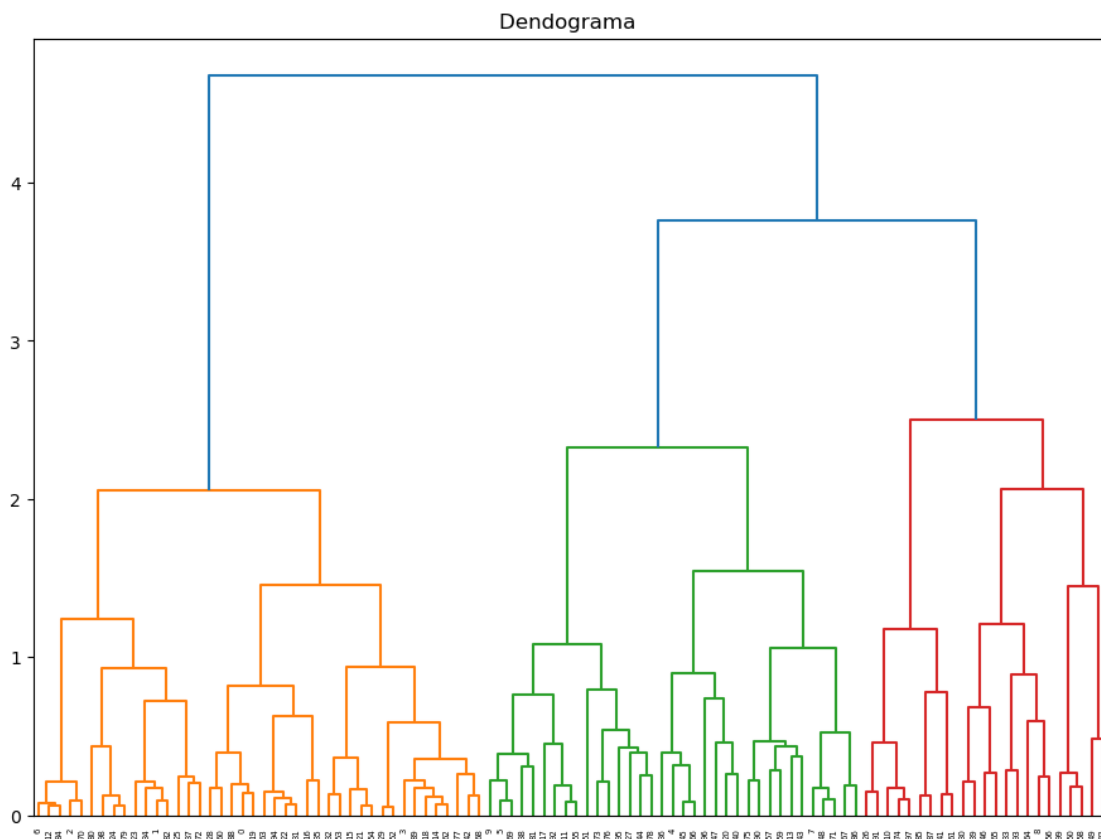
0	0.049171	0.055585	0.044895	0.036344
1	0.010327	0.108431	0.555065	0.072287
2	0.090902	0.138036	0.087535	0.111102
3	0.044101	0.074781	0.373904	0.032597
4	0.183617	0.898127	0.135717	0.171642

Después de que normalicé los datos, los resultados quedaron en forma de una matriz numérica. Es por eso que los convertí nuevamente en un DataFrame.

Esto hace que los datos sean más fáciles de leer y también permite seguir trabajando con las columnas por su nombre.

2.6 Construcción del dendrograma

```
[11]: #Construcción del dendrograma.
import scipy.cluster.hierarchy as shc
plt.figure(figsize=(11,8))
plt.title("Dendrograma ")
dend = shc.dendrogram(shc.linkage(data_scaled,method="ward"))
```



En esta parte construí el dendrograma utilizando el método de Ward. El dendrograma permite ver de manera gráfica cómo se van uniendo los clientes de acuerdo con qué tan similares son sus

características. Esta gráfica me sirve como una guía para decidir cuántos clústeres tiene sentido formar.

2.7 Identificación de los colores del dendrograma.

```
[12]: #Colores del dendrograma.
print(dend['color_list'])

['C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1',
'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1',
'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1', 'C1',
'C1', 'C1', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2',
'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2',
'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C2', 'C3', 'C3', 'C3',
'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C3',
'C3', 'C3', 'C3', 'C3', 'C3', 'C3', 'C0', 'C0']
```

En este paso revisé la lista de colores que se utilizaron para construir el dendrograma. Estos colores ayudan a distinguir de forma visual los diferentes grupos que aparecen en la gráfica.

2.8 Identificación de los colores únicos.

```
[13]: #Colores únicos en el dendrograma
colores_unicos = set(dend['color_list']) #Elementos no repetido de la lista
colores_unicos
```

```
[13]: {'C0', 'C1', 'C2', 'C3'}
```

En este paso me quedé solamente con los colores que son diferentes en el dendrograma. El resultado muestra cuatro colores distintos: C0, C1, C2 y C3. Esto nos ayuda en el siguiente paso a determinar cuántos grupos principales se están formando.

2.9 Número de clústeres.

```
[14]: #Número de clusters
num_clusters_optimo = len(colores_unicos)-1
num_clusters_optimo
```

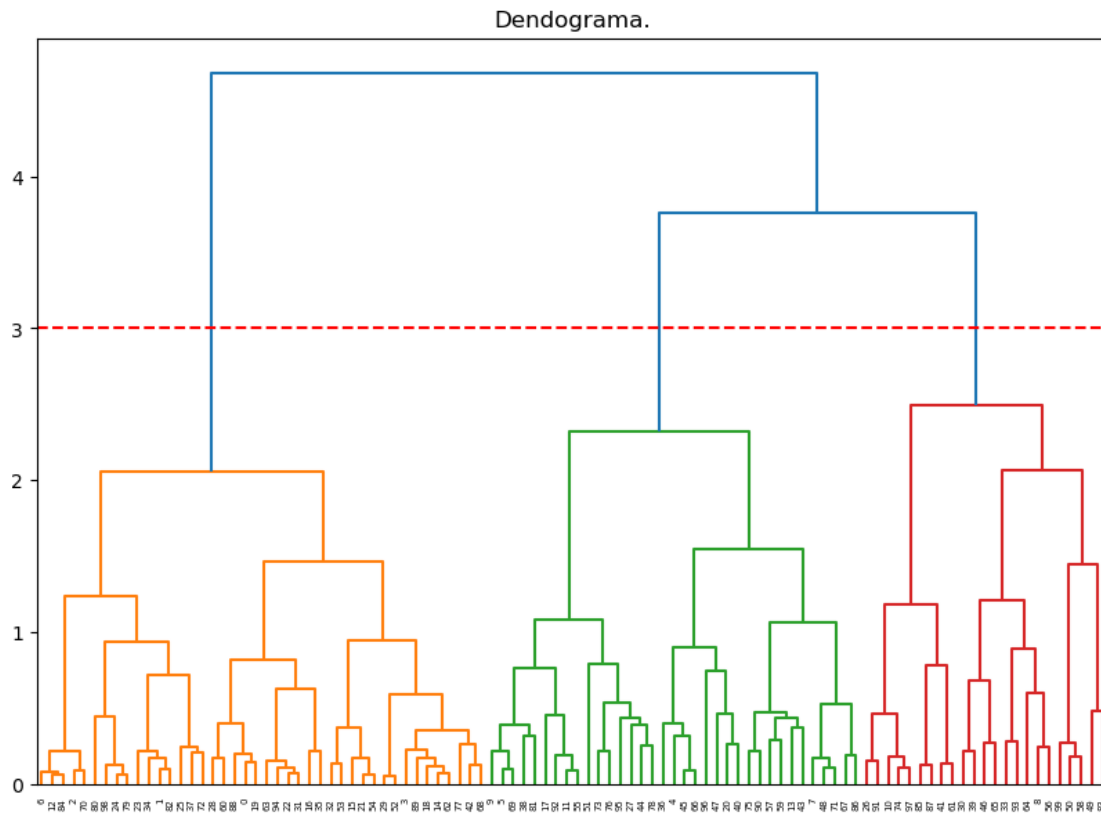
```
[14]: 3
```

Después conté los colores diferentes y resté uno, ya que uno de los colores corresponde a las uniones superiores del dendrograma. Con esto obtuve un resultado de 3 clusters.

2.10 Corte en el dendrograma.

```
[15]: #Corte en el dendrograma
plt.figure(figsize=(10,7))
plt.title("Dendrograma.")
dend = shc.dendrogram(shc.linkage(data_scaled, method='ward'))
plt.axhline(y=3,color='r',linestyle='--')
```

[15]: <matplotlib.lines.Line2D at 0x7e17f5b7eab0>



En esta gráfica agregué una línea roja horizontal en el valor 3. Esta línea sirve para ver de una forma más clara dónde se está haciendo el corte del dendrograma y cómo a partir de ese punto quedan separados los tres grupos.

2.11 Formación de los clústeres

```
[16]: #Formación de los Clusters
from sklearn.cluster import AgglomerativeClustering
cluster = AgglomerativeClustering(n_clusters=3, metric='euclidean',
    linkage='ward')
grupos = cluster.fit_predict(data_scaled)
grupos
```

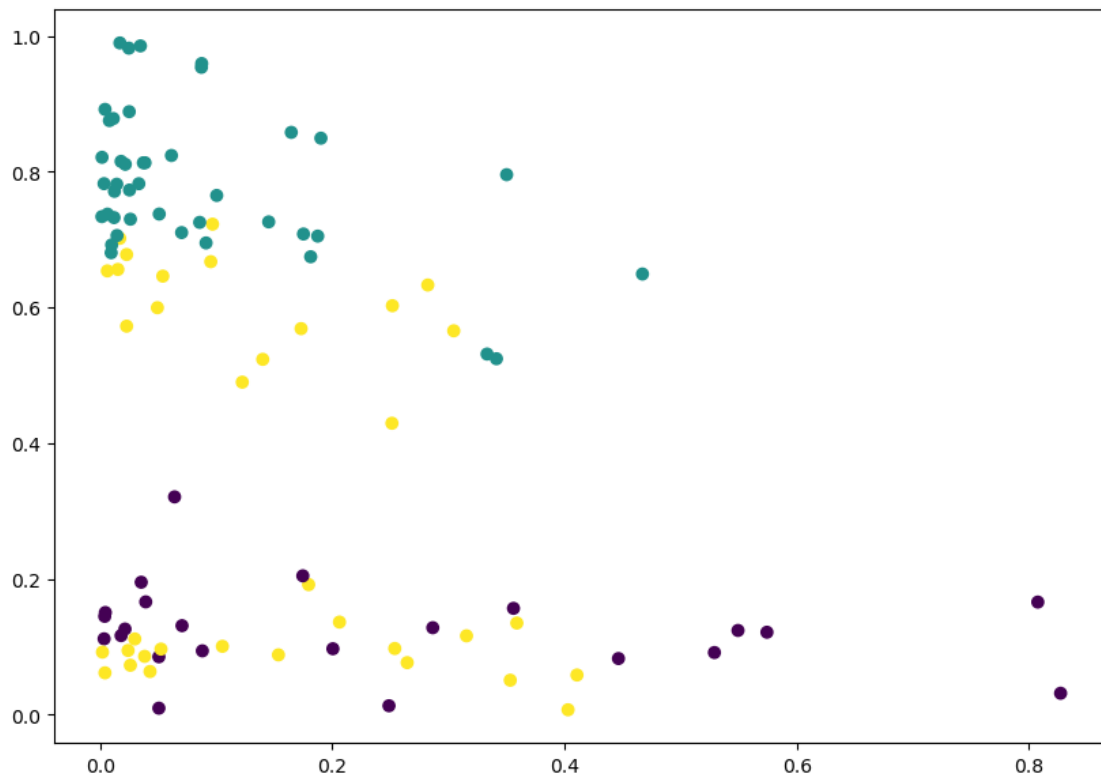
```
[16]: array([1, 1, 1, 1, 2, 2, 1, 2, 0, 2, 0, 2, 1, 2, 1, 1, 1, 2, 1, 1, 2, 1,
        1, 1, 1, 1, 0, 2, 1, 1, 0, 1, 1, 0, 1, 1, 2, 1, 2, 0, 2, 0, 1, 2,
        2, 2, 0, 2, 2, 0, 0, 2, 1, 1, 1, 2, 0, 2, 0, 2, 1, 0, 1, 1, 0, 0,
        2, 2, 1, 2, 1, 2, 1, 2, 0, 2, 2, 1, 2, 1, 1, 2, 1, 0, 1, 0, 2, 0,
        1, 1, 2, 0, 2, 0, 1, 2, 2, 0, 1, 0])
```

Estos números solamente identifican a qué grupo pertenece cada cliente.

2.12 Visualización de los clústeres.

```
[17]: #Visualización de los clusters
plt.figure(figsize=(10,7))
plt.scatter(data_scaled['Precio'],data_scaled['Durabilidad'],c = cluster.labels_)
```

```
[17]: <matplotlib.collections.PathCollection at 0x7e17f646f080>
```



En esta gráfica representé a los clientes utilizando las variables Precio y Durabilidad. Cada color corresponde a uno de los tres clústeres. Esto nos permite observar de forma visual cómo se están separando algunos de los clientes respecto a estas variables

2.13 Conversión de los datos para el ACP

```
[18]: #Conversión de datos para el ACP
campos = data_scaled.values
campos[:5]
```

```
[18]: array([[0.43826336, 0.00641361, 0.73756517, 0.50239946, 0.05130888,
         0.04917101, 0.05558462, 0.04489527, 0.03634379],
        [0.02323527, 0.03872544, 0.81323434, 0.08519598, 0.06454241,
         0.01032679, 0.10843125, 0.55506471, 0.0722875 ],
        [0.05723452, 0.08753514, 0.95951982, 0.01010021, 0.14476966,
```



```

0.09090188, 0.13803618, 0.08753514, 0.11110229],
[0.25885639, 0.00958727, 0.68069644, 0.56564916, 0.03451419,
0.04410146, 0.07478074, 0.37390368, 0.03259673],
[0.01197502, 0.17962533, 0.19160036, 0.15567529, 0.13571692,
0.18361701, 0.89812667, 0.13571692, 0.17164198]])

```

En este paso convertí el DataFrame de los datos normalizados en una matriz de valores. Esto se hizo porque estos valores se utilizarán en el Análisis de Componentes Principales. Nuevamente solo mostré un panorama general de los resultados, debido a la gran cantidad de valores obtenidos.

2.14 Análisis de Componentes Principales.

```

[20]: #ACP
from sklearn import decomposition
pca= decomposition.PCA(n_components=2)
pca.fit(campos)
campos = pca.transform(campos)
campos[:10]

```

```

[20]: array([[ 0.39993219, -0.06035374],
[ 0.40829583,  0.06402268],
[ 0.49703182,  0.35955286],
[ 0.33996675, -0.18434501],
[-0.58207751,  0.38797653],
[-0.1723403 ,  0.52962573],
[ 0.56528441,  0.33770585],
[-0.5335887 , -0.06994712],
[-0.09981604, -0.47165743],
[-0.22195207,  0.56503313]])

```

Aquí realicé el ACP utilizando solo dos componentes. La idea es resumir las variables originales en solamente dos nuevas componentes para que después pueda representar los resultados en una gráfica y se pueda observar mejor la separación de los grupos.

Debido a la gran cantidad de resultados, muestro únicamente el panorama general que permita visualizar de forma más clara la distribución de los grupos.

2.15 Representación gráfica de los clústeres con ACP.

```

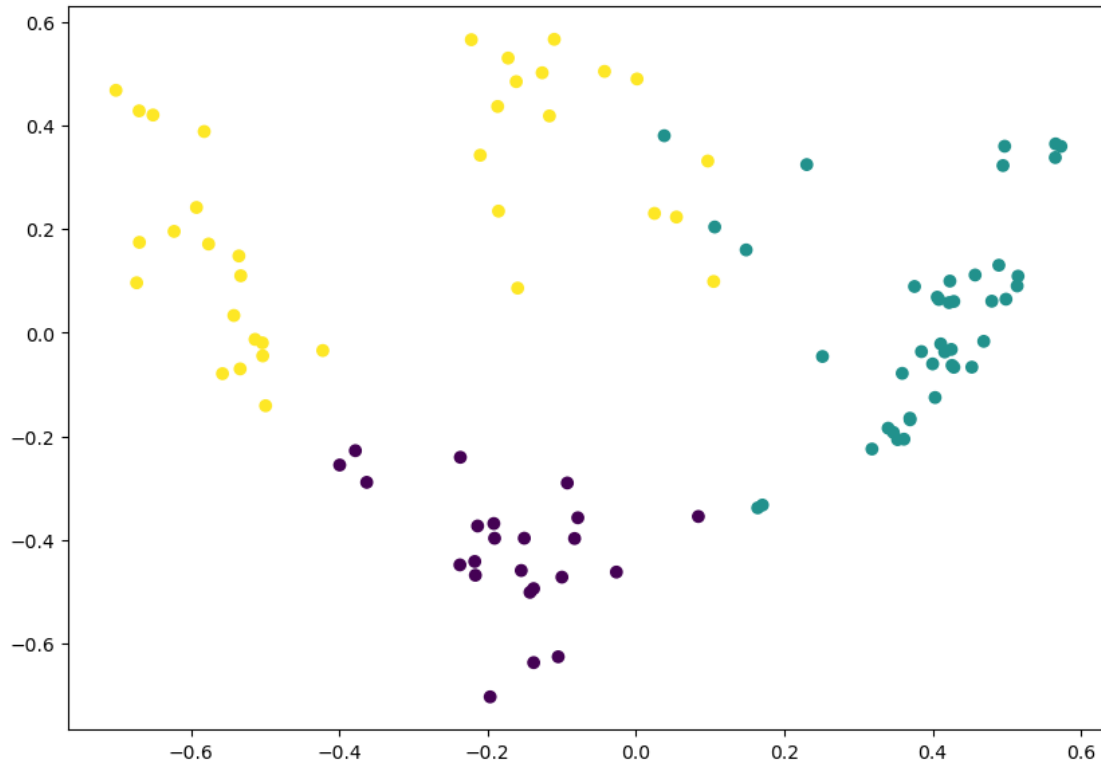
[21]: #Representación gráfica de los clusters
plt.figure(figsize=(10,7))
plt.scatter(campos[:,0], campos[:,1],c=cluster.labels_)

```

```

[21]: <matplotlib.collections.PathCollection at 0x7e17f57af3b0>

```



En esta gráfica se utilizaron los dos componentes obtenidos con el ACP. Cada punto representa a un cliente y cada color corresponde a un clúster.

2.16 Asignación de los clústeres a cada observación.

```
[23]: #Asignación de los clusters a cada observación
dataframe = pd.DataFrame(grupos, columns=['CJ'])
dataframe[:10]
```

```
[23]:   CJ
0    1
1    1
2    1
3    1
4    2
5    2
6    1
7    2
8    0
9    2
```

En este paso guardé la clasificación obtenida en una nueva columna llamada CJ. Así podemos saber fácilmente a cuál de los tres clústeres pertenece cada persona.

2.17 Unión de los datos con los clústeres.

```
[24]: #Unión de los datos con los clusters
dataframe2 = pd.concat([data_sin_nombres, dataframe], axis=1, join='inner')
dataframe2[:10]
```

```
[24]:  Velocidad Entrega  Precio  Durabilidad  Imagen Producto  Valor Educativo  \
0                205      3          345          235          24
1                 9     15          315           33          25
2                17     26          285           3          43
3               135      5          355          295          18
4                 3     45           48           39          34
5                95    165          395           24          26
6                23     12          475           33          35
7                65     21           31          255          28
8               275      8           47          235          35
9                 2    175          325           3          37

      Servicio Retorno  Tamano Paquete  Calidad Producto  Numero Estrellas  CJ
0                 23          26          21          17  1
1                  4          42          215          28  1
2                 27          41           26          33  1
3                 23          39          195          17  1
4                 46         225           34          43  2
5                 19         485           22           3  2
6                 45          38           29          31  1
7                 22         345          215          29  2
8                  3          38           27          48  0
9                 32         435           27          39  2
```

Aquí uní las variables originales con la columna que contiene el número del clúster. De esta manera ahora tenemos las características de cada cliente y también el grupo al que pertenece.

2.18 Incorporación de los nombres.

```
[26]: #Incorporación de los nombres
dataframe2.insert(0, 'Nombre', data.iloc[:, 0].values)
dataframe2[:10]
```

```
[26]:  Nombre  Velocidad Entrega  Precio  Durabilidad  Imagen Producto  \
0    Adam          205      3          345          235
1    Anna           9     15          315           33
2  Bernard          17     26          285           3
3   Edward         135      5          355          295
4   Emilia           3     45           48           39
5   Fabian          95    165          395           24
6   Philip          23     12          475           33
7   Frank          65     21           31          255
```

8	Xavier	275	8	47	235
9	Gabriel	2	175	325	3

	Valor Educativo	Servicio Retorno	Tamano Paquete	Calidad Producto	\
0	24	23	26	21	
1	25	4	42	215	
2	43	27	41	26	
3	18	23	39	195	
4	34	46	225	34	
5	26	19	485	22	
6	35	45	38	29	
7	28	22	345	215	
8	35	3	38	27	
9	37	32	435	27	

	Numero Estrellas	CJ
0	17	1
1	28	1
2	33	1
3	17	1
4	43	2
5	3	2
6	31	1
7	29	2
8	48	0
9	39	2

Al inicio del análisis separé los nombres de los clientes para trabajar únicamente con las variables numéricas durante la normalización y la formación de los clústeres. Una vez obtenida la clasificación, incorporaré los nombres nuevamente para identificar a qué cliente corresponde cada observación.

2.19 Identificación del clúster de los clientes.

```
[59]: #Identificación del cluster de los clientes
clientes = ['Salome', 'Stephania', 'Lydia']

dataframe2[ dataframe2['Nombre'].isin(clientes)][['Nombre', 'CJ']]
```

```
[59]:      Nombre  CJ
54     Lydia   1
81     Salome   2
89  Stephania   1
```

Aquí busqué específicamente a Lydia, Salome y Stephania para saber en qué clúster quedó cada una. Lydia y Stephania quedaron en el clúster 1, mientras que Salome quedó en el clúster 2.

2.20 Clasificación por clústeres

```
[60]: #Calsificación por clusters
cluster_1 = dataframe2[dataframe2['CJ'] == 0]
cluster_2 = dataframe2[dataframe2['CJ'] == 1]

print("CLUSTER 1")
display(cluster_1)

print("CLUSTER 2")
display(cluster_2)
```

CLUSTER 1

	Nombre	Velocidad Entrega	Precio	Durabilidad	Imagen Producto \
8	Xavier	275	8	47	235
10	Marisol	12	8	44	24
26	Florent	12	75	33	24
30	Herman	15	16	3	265
33	Isadore	17	185	32	285
39	John	12	1	32	225
41	Josephine	295	45	48	39
46	Lawrence	155	95	5	225
49	Leonid	27	105	4	15
50	Lesia	185	35	41	3
56	Maksym	245	22	37	345
58	Margaret	215	9	38	27
61	Marianna	255	95	46	29
64	Martha	55	1	36	235
65	Martin	185	7	45	225
74	Michaelina	15	1	33	33
83	Sandra	115	185	38	25
85	Savina	125	155	35	21
87	Sophia	105	175	37	24
91	Sylvan	24	85	38	21
93	Tamara	95	135	25	245
97	Teofila	1	14	26	25
99	Teresa	125	9	45	25

	Valor Educativo	Servicio Retorno	Tamano Paquete	Calidad Producto \
8	35	3	38	27
10	2	28	29	215
26	19	25	36	185
30	31	3	4	165
33	35	34	42	19
39	21	22	44	165
41	34	46	225	335
46	26	32	19	245
49	38	14	26	19

50	21	25	26	25
56	46	4	48	31
58	31	25	22	28
61	36	23	225	305
64	16	32	5	17
65	26	23	34	245
74	24	27	41	205
83	3	25	37	22
85	28	22	45	185
87	28	23	36	215
91	33	14	29	275
93	22	25	41	18
97	24	27	42	185
99	22	3	3	22

	Numero Estrellas	CJ
8	48	0
10	17	0
26	21	0
30	28	0
33	32	0
39	13	0
41	43	0
46	4	0
49	38	0
50	26	0
56	47	0
58	31	0
61	45	0
64	25	0
65	3	0
74	26	0
83	22	0
85	28	0
87	21	0
91	24	0
93	21	0
97	23	0
99	18	0

CLUSTER 2

	Nombre	Velocidad	Entrega	Precio	Durabilidad	Imagen	Producto	\
0	Adam		205	3	345		235	
1	Anna		9	15	315		33	
2	Bernard		17	26	285		3	
3	Edward		135	5	355		295	
6	Philip		23	12	475		33	
12	Irene		14	7	405		19	

14	Isidore	235	65	495	335
15	Joseph	17	1	485	235
16	Eugene	16	205	285	255
18	Eunice	265	7	485	305
19	Eva	235	65	495	335
21	Fedir	17	2	415	125
22	Felix	15	2	455	355
23	Fialka	12	75	335	24
24	Flavia	255	7	435	24
25	Flora	23	105	395	29
28	Hannah	175	14	495	175
29	Helen	205	185	295	275
31	Hilary	14	19	445	345
32	Lourdes	26	1	465	295
34	Ivan	12	5	385	17
35	Jacob	9	165	375	225
37	Jervis	2	45	455	27
42	Judith	245	115	465	225
52	Louise	225	205	315	295
53	Lubomyr	14	12	335	245
54	Lydia	19	4	435	145
60	Marian	155	95	495	225
62	Markian	205	55	465	275
63	Marko	15	19	275	245
68	Maura	265	85	425	185
70	Maximillian	18	27	295	31
72	Methodius	18	11	495	24
77	Mykyta	165	13	485	165
79	Myroslav	225	8	435	23
80	Myroslava	275	9	435	19
82	Samuel	8	14	305	32
84	Sarah	13	15	425	3
88	Stephan	145	6	365	305
89	Stephania	215	125	465	315
94	Theodore	2	25	335	225
98	Teon	155	11	335	34

	Valor Educativo	Servicio Retorno	Tamano Paquete	Calidad Producto	\
0	24	23	26	21	
1	25	4	42	215	
2	43	27	41	26	
3	18	23	39	195	
6	35	45	38	29	
12	21	14	33	22	
14	3	26	34	295	
15	27	17	24	235	
16	36	29	31	22	
18	33	39	34	295	

19	3	26	34	3
21	12	17	26	165
22	35	34	42	26
23	19	25	36	185
24	33	26	19	245
25	34	28	235	295
28	31	17	27	27
29	39	3	42	255
31	33	32	41	25
32	37	24	23	305
34	17	11	31	205
35	25	24	38	18
37	24	26	365	255
42	36	13	31	295
52	43	34	44	275
53	25	26	46	185
54	16	21	28	185
60	26	31	19	24
62	25	27	37	265
63	34	26	3	21
68	35	19	24	215
70	45	29	42	27
72	29	19	245	245
77	29	15	26	255
79	31	21	34	255
80	36	21	245	225
82	23	38	41	205
84	28	28	34	28
88	2	25	4	2
89	34	4	37	305
94	22	21	25	2
98	26	29	42	215

	Numero	Estrellas	CJ
0	17	1	
1	28	1	
2	33	1	
3	17	1	
6	31	1	
12	24	1	
14	39	1	
15	34	1	
16	23	1	
18	39	1	
19	4	1	
21	2	1	
22	4	1	
23	21	1	

24	34	1
25	34	1
28	34	1
29	31	1
31	38	1
32	45	1
34	2	1
35	24	1
37	31	1
42	38	1
52	35	1
53	17	1
54	24	1
60	39	1
62	32	1
63	21	1
68	43	1
70	35	1
72	36	1
77	32	1
79	41	1
80	44	1
82	26	1
84	38	1
88	19	1
89	45	1
94	16	1
98	27	1

En esta parte separé a los clientes de acuerdo con el clúster al que fueron asignados y mostré sus características para poder comparar a las personas que quedaron dentro de un mismo grupo.

En el código original solamente se muestran las observaciones correspondientes a $CJ = 0$ y $CJ = 1$. Por esa razón aquí se presentan únicamente las salidas de CLUSTER 1 y CLUSTER 2, tal como aparecen en el análisis original.

2.21 Dendrograma con nombres

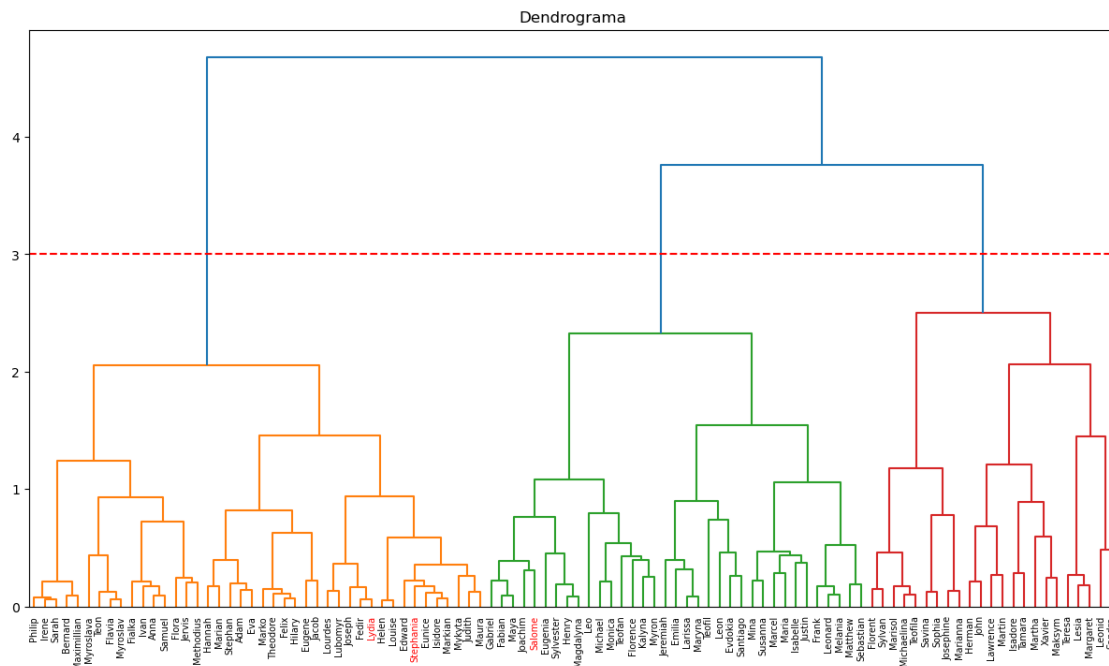
```
[72]: #Dendrograma con nombres
plt.figure(figsize=(15,8))
plt.title("Dendrograma")
dend = shc.dendrogram(shc.linkage(data_scaled, method='ward'),labels=data.iloc[:
    ↪,0].values,
                        leaf_rotation=90,leaf_font_size=7)
plt.axhline(y=3, color='r', linestyle='--')

buscar = ['Salome', 'Stephania', 'Lydia']
for nombre in plt.gca().get_xticklabels():
```

```

if nombre.get_text() in buscar:
    nombre.set_color('red')
plt.show()

```



Finalmente volví a construir el dendrograma, pero ahora agregué los nombres de los clientes. También marqué en rojo a Salome, Stephania y Lydia para poder ubicarlas fácilmente y observar en qué parte del dendrograma quedó cada una.

Tomando en cuenta el dendrograma recomendaré productos de acuerdo con las personas que quedaron más cercas entre sí.

A Salomé le recomendaría los mismos productos que compró Joachim.

A Stephania le recomendaría productos similares a los que compraron Eunice, Isidore y Markian, porque en el dendrograma Stephania se une primero al grupo formado por estos clientes

A Lydia le recomendaría los mismos productos que compró Fedir

1